

Scanpy Profiling Report

BMI500 HW3 - Shashank Chauhan

Pbmc3k –

1. Coarse-grain (gtime -v) -

```
Command being timed: "python scanpy_pbmc.py --data-dir data --data-set pbmc3k --out-dir data"
User time (seconds): 27.18
System time (seconds): 1.99
Percent of CPU this job got: 118%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:24.64
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 716912
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 6821
Minor (reclaiming a frame) page faults: 186409
Voluntary context switches: 3692
Involuntary context switches: 39368
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 16384
Exit status: 0
○ (bmi500) shashankchauhan@Shashanks-MacBook-Air-7 bmi500 %
```

2. Instrumented per-section timing -

```

[TIMING] I/O (read 10x_mtx): 0.125s
filtered out 19024 genes that are detected in less than 3 cells
[TIMING] basic filtering (filter_cells/filter_genes): 0.053s
normalizing counts per cell
  finished (0:00:00)
[TIMING] normalization (normalize_total + log1p): 0.039s
extracting highly variable genes
  finished (0:00:00)
--> added
  'highly_variable', boolean vector (adata.var)
  'means', float vector (adata.var)
  'dispersions', float vector (adata.var)
  'dispersions_norm', float vector (adata.var)
[TIMING] highly_variable_genes + filtering: 0.058s
/opt/anaconda3/envs/bmi500/lib/python3.12/site-packages/scanpy/preprocessing/_scale.py:316: UserWarning: Received a view of an AnnData. Making a copy.
  view_to_actual(adata)
... as `zero_center=True`, sparse input is densified and may lead to large memory consumption
[TIMING] scale: 0.036s
[TIMING] report adata (no-op): 0.000s
computing PCA
  with n_comps=30
  finished (0:00:00)
[TIMING] PCA: 0.340s
computing neighbors
  using 'X_pca' with n_pcs = 30
  finished: added to '.uns['neighbors']'
  '.obsp['distances']', distances for each pair of neighbors
  '.obsp['connectivities']', weighted adjacency matrix (0:00:03)
[TIMING] neighbors: 3.593s
[TIMING] paga fix (no-op): 0.000s
running Louvain clustering
  using the "louvain" package of Traag (2017)
  finished: found 4 clusters and added
  'louvain', the cluster labels (adata.obs, categorical) (0:00:00)
[TIMING] clustering (louvain): 0.144s
computing UMAP
  finished: added
  'X_umap', UMAP coordinates (adata.obsm) (0:00:07)
[TIMING] UMAP: 7.080s
[TIMING] write h5ad: 0.158s
ranking genes
  finished: added to '.uns['rank_genes_groups']'
  'names', sorted np.recarray to be indexed by group ids
  'scores', sorted np.recarray to be indexed by group ids
  'logfoldchanges', sorted np.recarray to be indexed by group ids
  'pvals', sorted np.recarray to be indexed by group ids
  'pvals_adj', sorted np.recarray to be indexed by group ids (0:00:02)
[TIMING] rank_genes_groups: 2.737s
Tue Sep 15 19:23:27 2026    data/pbmc3k_rank_genes_groups.pstats

42440 function calls (41199 primitive calls) in 2.728 seconds

```

3. Fine-grain – cProfile on rank_genes_groups -

```

Ordered by: cumulative time
List reduced from 1033 to 20 due to restriction <20>

ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
24/1    0.000    0.000    2.728    2.728    {built-in method builtins.exec}
1       0.000    0.000    2.728    2.728    __init__.py:83(fn_compatible)
1       0.000    0.000    2.728    2.728    _rank_genes_groups.py:469(rank_genes_groups)
1       0.003    0.003    2.678    2.678    _rank_genes_groups.py:398(compute_statistics)
5       0.012    0.002    2.635    0.527    _rank_genes_groups.py:277(wilcoxon)
5       0.008    0.002    2.386    0.477    _rank_genes_groups.py:45(_ranks)
4       0.000    0.000    2.348    0.587    generic.py:10012(rank)
4       0.000    0.000    2.348    0.587    generic.py:10127(ranker)
4       2.347    0.587    2.347    0.587    algorithms.py:1067(rank)
1       0.003    0.003    0.109    0.109    _rank_genes_groups.py:174(_basic_stats)
12      0.000    0.000    0.108    0.009    _index.py:29(__getitem__)
8       0.000    0.000    0.098    0.012    _csc.py:140(_get_arrayXslice)
8       0.000    0.000    0.098    0.012    _compressed.py:651(_minor_index_fancy)
8       0.009    0.011    0.089    0.011    {built-in method scipy.sparse._sparsetools.csr_column_index2}
60/59   0.000    0.000    0.083    0.001    indexing.py:1177(__getitem__)
41      0.000    0.000    0.080    0.002    indexing.py:1690(_getitem_tuple)
57      0.000    0.000    0.079    0.001    indexing.py:1720(_getitem_axis)
23      0.000    0.000    0.078    0.003    indexing.py:1206(_getbool_axis)
23      0.000    0.000    0.077    0.003    generic.py:4164(_take_with_is_copy)
23      0.000    0.000    0.076    0.003    generic.py:4049(take)

```

Same data - visualized and sorted by tottime -

Search:

	ncalls	tottime	pcall	cumtime	pcall	filename:lineno(function)
4	2.384	0.596	2.384	0.5961	algorithms.py:1067(rank)	
8	0.08975	0.01122	0.08975	0.01122	~O(<built-in method scipy.sparse._sparsetools.csr_column_index2>)	
53	0.08689	0.001639	0.08736	0.001648	take.py:120(_take_nd_ndarray)	
270	0.04213	0.000156	0.04213	0.000156	~O(<method 'reduce' of 'numpy.ufunc' objects>)	
1	0.02359	0.02359	0.02369	0.02369	__init__.py:781(_check_nonnegative_integers_in_mem)	
22	0.01349	0.0006131	0.01362	0.0006189	missing.py:261(_isna_array)	
5	0.01272	0.002545	2.697	0.5395	_rank_genes_groups.py:277(wilcoxon)	
23	0.01096	0.0004766	0.01096	0.0004766	~O(<method 'argsort' of 'numpy.ndarray' objects>)	
22	0.0096	0.0004364	0.0096	0.0004364	~O(<built-in method numpy.zeros>)	
4	0.008538	0.002134	0.008538	0.002134	~O(<built-in method scipy.sparse._sparsetools.get_csr_submatrix>)	
4	0.008173	0.002043	0.008173	0.002043	~O(<built-in method scipy.sparse._sparsetools.csr_todense>)	
8	0.00776	0.00097	0.00776	0.00097	~O(<built-in method scipy.sparse._sparsetools.csr_column_index1>)	
8	0.005974	0.0007468	0.006146	0.0007683	_utils.py:47(sparse_mean_variance_axis)	
45	0.005175	0.000115	0.00612	0.000136	function_base.py:5369(insert)	
23	0.00488	0.0002122	0.00488	0.0002122	~O(<method 'read' of '_io.BufferedReader' objects>)	
1	0.004185	0.004185	0.004185	0.004185	~O(<built-in method scipy.sparse._sparsetools.csr_eliminate_zeros>)	
1	0.003793	0.003793	2.76	2.76	_rank_genes_groups.py:398(compute_statistics)	
328	0.003497	1.066e-05	0.003514	1.071e-05	~O(<built-in method __new__ of type object at 0x104803330>)	
1	0.003207	0.003207	0.1086	0.1086	_rank_genes_groups.py:174(_basic_stats)	
169/5	0.003089	0.0006179	0.003404	0.0006808	~O(<built-in method _abc._abc_subclasscheck>)	
16	0.003017	0.0001886	0.06359	0.003975	frame.py:11462(_reduce)	
23	0.002836	0.0001233	0.002836	0.0001233	~O(<built-in method marshal.load>)	
5	0.002325	0.0004649	2.416	0.4833	_rank_genes_groups.py:45(_ranks)	
92	0.00191	2.076e-05	0.002039	2.216e-05	base.py:5385(_getitem_)	
4	0.001663	0.0004157	0.001663	0.0004157	_continuous_distns.py:370(_norm_cdf)	
1	0.001583	0.001583	0.00198	0.00198	series.py:3706(sort_values)	

Pbmc6k –

1. Coarse-grain (gtime -v) -

```
Command being timed: "python scanpy_pbmc.py --data-dir data --data-set pbmc6k --out-dir data"
User time (seconds): 37.60
System time (seconds): 1.95
Percent of CPU this job got: 116%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:33.87
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 783760
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 6996
Minor (reclaiming a frame) page faults: 190263
Voluntary context switches: 3802
Involuntary context switches: 43578
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 16384
Exit status: 0
o (bmi500) shashankchauhan@Shashanks-MacBook-Air-7 bmi500 %
```

2. Instrumented per-section timing -

```
[TIMING] I/O (read_10x_mtx): 0.155s
filtered out 1 cells that have less than 200 genes expressed
filtered out 18395 genes that are detected in less than 3 cells
[TIMING] basic filtering (filter_cells/filter_genes): 0.129s
normalizing counts per cell
  finished (0:00:00)
[TIMING] normalization (normalize_total + log1p): 0.062s
extracting highly variable genes
  finished (0:00:00)
--> added
  'highly_variable', boolean vector (adata.var)
  'means', float vector (adata.var)
  'dispersions', float vector (adata.var)
  'dispersions_norm', float vector (adata.var)
[TIMING] highly_variable_genes + filtering: 0.072s
/opt/anaconda3/envs/bmi500/lib/python3.12/site-packages/scanpy/preprocessing/_scale.py:316: UserWarning: Received a view of an AnnData. Making a copy.
  view_to_actual(adata)
... as `zero_center=True`, sparse input is densified and may lead to large memory consumption
[TIMING] scale: 0.068s
[TIMING] report adata (no-op): 0.000s
computing PCA
  with n_comps=30
  finished (0:00:00)
[TIMING] PCA: 0.420s
computing neighbors
  using 'X_pca' with n_pcs = 30
  finished: added to '.uns['neighbors']'
  '.obs['distances']', distances for each pair of neighbors
  '.obs['connectivities']', weighted adjacency matrix (0:00:03)
[TIMING] neighbors: 3.601s
[TIMING] paga fix (no-op): 0.000s
running Louvain clustering
  using the "louvain" package of Traag (2017)
  finished: found 7 clusters and added
  'louvain', the cluster labels (adata.obs, categorical) (0:00:00)
[TIMING] clustering (louvain): 0.378s
computing UMAP
  finished: added
  'X_umap', UMAP coordinates (adata.obsm) (0:00:13)
[TIMING] UMAP: 13.009s
[TIMING] write h5ad: 0.143s
ranking genes
  finished: added to '.uns['rank_genes_groups']'
  'names', sorted np.recarray to be indexed by group ids
  'scores', sorted np.recarray to be indexed by group ids
  'logfoldchanges', sorted np.recarray to be indexed by group ids
  'pvals', sorted np.recarray to be indexed by group ids
  'pvals_adj', sorted np.recarray to be indexed by group ids (0:00:05)
[TIMING] rank_genes_groups: 5.696s
Tue Sep 15 19:36:06 2026    data/pbmc6k_rank_genes_groups.pstats

80418 function calls (78545 primitive calls) in 5.688 seconds
```

3. Fine-grain - cProfile on rank_genes_groups -

```
Ordered by: cumulative time
List reduced from 1033 to 20 due to restriction <20>

ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
24/1    0.000    0.000    5.688    5.688  {built-in method builtins.exec}
1       0.001    0.001    5.688    5.688  __init__.py:83(fn_compatible)
1       0.000    0.000    5.687    5.687  _rank_genes_groups.py:469(rank_genes_groups)
1       0.005    0.005    5.610    5.610  _rank_genes_groups.py:398(compute_statistics)
8       0.021    0.003    5.548    0.694  _rank_genes_groups.py:277(wilcoxon)
9       0.005    0.001    4.990    0.554  _rank_genes_groups.py:45(_ranks)
8       0.000    0.000    4.929    0.616  generic.py:10012(rank)
8       0.000    0.000    4.929    0.616  generic.py:10127(ranker)
8       4.927    0.616    4.928    0.616  algorithms.py:1067(rank)
22      0.000    0.000    0.252    0.011  _index.py:29(__getitem__)
1       0.005    0.005    0.247    0.247  _rank_genes_groups.py:174(_basic_stats)
14      0.000    0.000    0.229    0.016  _csc.py:140(_get_arrayXslice)
14      0.000    0.000    0.229    0.016  _compressed.py:651(_minor_index_fancy)
14      0.205    0.015    0.205    0.015  {built-in method scipy.sparse._sparsetools.csr_column_index2}
155/154 0.001    0.000    0.183    0.001  indexing.py:1177(__getitem__)
96      0.000    0.000    0.178    0.002  indexing.py:1690(_getitem_tuple)
152     0.001    0.000    0.176    0.001  indexing.py:1720(_getitem_axis)
63      0.000    0.000    0.172    0.003  indexing.py:1206(_getbool_axis)
63      0.000    0.000    0.170    0.003  generic.py:4164(_take_with_is_copy)
56      0.000    0.000    0.170    0.003  indexing.py:1005(_getitem_tuple_same_dim)
```

Same data - visualized and sorted by tottime -

	ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
8	4.927	0.6159	4.928	0.616	algorithms.py:1067(rank)	
14	0.2055	0.01468	0.2055	0.01468	~/0(<built-in method scipy.sparse._spsparsetools.csr_column_index2>)	
108	0.1608	0.001489	0.1616	0.001497	take.py:120(_take_nd_ndarray)	
693	0.0681	9.827e-05	0.0681	9.827e-05	~/0(<method 'reduce' of 'numpy.ufunc' objects>)	
1	0.04375	0.04375	0.044	0.044	__init__.py:781(_check_nonnegative_integers_in_mem)	
35	0.02442	0.0006977	0.02442	0.0006977	~/0(<built-in method numpy.zeros>)	
62	0.02387	0.000385	0.02403	0.0003876	missing.py:261(_isna_array)	
14	0.02145	0.001532	0.02145	0.001532	~/0(<built-in method scipy.sparse._spsparsetools.csr_column_index1>)	
8	0.02066	0.002583	0.02066	0.002583	~/0(<built-in method scipy.sparse._spsparsetools.get_csr_submatrix>)	
8	0.02051	0.002564	5.548	0.6935	_rank_genes_groups.py:277(wilcoxon)	
35	0.01918	0.0005479	0.01918	0.0005479	~/0(<method 'argsort' of 'numpy.ndarray' objects>)	
14	0.01105	0.0007889	0.01127	0.0008046	_utils.py:47(sparse_mean_variance_axis)	
8	0.007417	0.0009271	0.007417	0.0009271	~/0(<built-in method scipy.sparse._spsparsetools.csr_todense>)	
1	0.007385	0.007385	0.007385	0.007385	~/0(<built-in method scipy.sparse._spsparsetools.csr_eliminate_zeros>)	
56	0.005761	0.0001029	0.1105	0.001974	frame.py:11462(_reduce)	
632	0.005738	9.079e-06	0.005751	9.099e-06	~/0(<built-in method __new__ of type object at 0x10475f330>)	
9	0.005406	0.0006006	4.99	0.5544	_rank_genes_groups.py:45(_ranks)	
1	0.005168	0.005168	0.2469	0.2469	_rank_genes_groups.py:174(_basic_stats)	
1	0.005079	0.005079	5.61	5.61	_rank_genes_groups.py:398(compute_statistics)	
23	0.004567	0.0001986	0.004567	0.0001986	~/0(<method 'read' of '_io.BufferedReader' objects>)	
12776/12746	0.002819	2.211e-07	0.007424	5.825e-07	~/0(<built-in method builtins.isinstance>)	
7	0.00248	0.0003543	0.00248	0.0003543	_continuous_distns.py:370(_norm_cdf)	
23	0.002322	0.000101	0.002322	0.000101	~/0(<built-in method marshal.loads>)	
2/1	0.002022	0.002022	0.002352	0.002352	algorithms.py:457(isin)	
169/5	0.001828	0.0003656	0.00273	0.0005461	~/0(<built-in method _abc._abc_subclasscheck>)	
1	0.001805	0.001805	0.002139	0.002139	series.py:3706(sort_values)	

Pbmc10k -

1. Coarse-grain (gtime -v) -

```

Command being timed: "python scanpy_pbmc.py --data-dir data --data-set pbmc10k --out-dir data"
User time (seconds): 71.91
System time (seconds): 5.27
Percent of CPU this job got: 109%
Elapsed (wall clock) time (h:mm:ss or m:ss): 1:10.24
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 1028896
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 7628
Minor (reclaiming a frame) page faults: 910842
Voluntary context switches: 3938
Involuntary context switches: 46582
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 16384
Exit status: 0
○ (bmi500) shashankchauhan@Shashanks-MacBook-Air-7 bmi500 %

```

2. Instrumented per-section timing -

```
... reading from cache file cache/data-pbmc10k-filtered_gene_bc_matrices-matrix.h5ad
[TIMING] I/O (read 10x_mtx): 0.510s
filtered out 232 cells that have less than 200 genes expressed
filtered out 13246 genes that are detected in less than 3 cells
[TIMING] basic filtering (filter_cells/filter_genes): 0.522s
normalizing counts per cell
  finished (0:00:00)
[TIMING] normalization (normalize_total + log1p): 0.420s
extracting highly variable genes
  finished (0:00:00)
--> added
  'highly_variable', boolean vector (adata.var)
  'means', float vector (adata.var)
  'dispersions', float vector (adata.var)
  'dispersions_norm', float vector (adata.var)
[TIMING] highly_variable_genes + filtering: 0.312s
/opt/anaconda3/envs/bmi500/lib/python3.12/site-packages/scanpy/preprocessing/_scale.py:316: UserWarning: Received a view of an AnnData. Making a copy.
  view_to_actual(adata)
... as 'zero_center=True', sparse input is densified and may lead to large memory consumption
[TIMING] scale: 0.127s
[TIMING] report adata (no-op): 0.000s
computing PCA
  with n_comps=30
  finished (0:00:00)
[TIMING] PCA: 0.918s
computing neighbors
  using 'X_pca' with n_pcs = 30
  finished: added to '.uns['neighbors']'
  '.obsp['distances']', distances for each pair of neighbors
  '.obsp['connectivities']', weighted adjacency matrix (0:00:24)
[TIMING] neighbors: 24.293s
[TIMING] paga fix (no-op): 0.000s
running Louvain clustering
  using the "louvain" package of Traag (2017)
  finished: found 15 clusters and added
  'louvain', the cluster labels (adata.obs, categorical) (0:00:01)
[TIMING] clustering (louvain): 1.221s
computing UMAP
  finished: added
  'X_umap', UMAP coordinates (adata.obsm) (0:00:11)
[TIMING] UMAP: 11.368s
[TIMING] write h5ad: 0.516s
ranking genes
  finished: added to '.uns['rank_genes_groups']'
  'names', sorted np.recarray to be indexed by group ids
  'scores', sorted np.recarray to be indexed by group ids
  'logfoldchanges', sorted np.recarray to be indexed by group ids
  'pvals', sorted np.recarray to be indexed by group ids
  'pvals_adj', sorted np.recarray to be indexed by group ids (0:00:19)
[TIMING] rank_genes_groups: 19.673s
Tue Sep 15 19:46:04 2026    data/pbmc10k_rank_genes_groups.pstats

      306107 function calls (300521 primitive calls) in 19.664 seconds
```

3. Fine-grain - cProfile on rank_genes_groups -

```
Ordered by: cumulative time
List reduced from 1064 to 20 due to restriction <20>

ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
25/1    0.000    0.000    19.664    19.664  {built-in method builtins.exec}
1       0.002    0.002    19.663    19.663  __init__.py:83(fn_compatible)
1       0.001    0.001    19.323    19.323  _rank_genes_groups.py:469(rank_genes_groups)
1       0.032    0.032    19.283    19.283  _rank_genes_groups.py:398(compute_statistics)
25      0.662    0.026    15.509    0.620  _rank_genes_groups.py:45(_ranks)
24      0.000    0.000    14.599    0.608  generic.py:10012(rank)
24      0.001    0.000    14.599    0.608  generic.py:10127(ranker)
24      14.593    0.608    14.594    0.608  algorithms.py:1067(rank)
16      0.051    0.003    11.659    0.729  _rank_genes_groups.py:277(wilcoxon)
1       0.057    0.057    2.465    2.465  _rank_genes_groups.py:174(_basic_stats)
54      0.000    0.000    2.419    0.045  _index.py:29(__getitem__)
30      0.000    0.000    2.295    0.076  _csc.py:140(_get_arrayXslice)
30      0.001    0.000    2.295    0.076  _compressed.py:651(_minor_index_fancy)
30      2.042    0.068    2.042    0.068  {built-in method scipy.sparse._sparsetools.csr_column_index2}
803/802 0.003    0.000    0.729    0.001  indexing.py:1177(__getitem__)
440     0.002    0.000    0.700    0.002  indexing.py:1690(_getitem_tuple)
800     0.003    0.000    0.699    0.001  indexing.py:1720(_getitem_axis)
367     0.001    0.000    0.676    0.002  indexing.py:1206(_getbool_axis)
360     0.002    0.000    0.675    0.002  indexing.py:1005(_getitem_tuple_same_dim)
367     0.001    0.000    0.664    0.002  generic.py:4164(_take_with_is_copy)
```

Same data - visualized and sorted by tottime -

	ncalls		tottime		percall		cumtime		percall		filename:lineno(function)
24			14.59		0.6081		14.59		0.6081		algorithms.py:1067(rank)
30			2.042		0.06806		2.042		0.06806		~0(<built-in method scipy.sparse._sparsetools.csr_column_index2>)
25			0.6622		0.02649		15.51		0.6204		_rank_genes_groups.py:45(_ranks)
452			0.618		0.001367		0.622		0.001376		take.py:120(_take_nd_ndarray)
1			0.2794		0.2794		0.2834		0.2834		__init__.py:781(_check_nonnegative_integers_in_mem)
30			0.2448		0.008161		0.2448		0.008161		~0(<built-in method scipy.sparse._sparsetools.csr_column_index1>)
3597			0.1996		5.549e-05		0.1996		5.549e-05		~0(<method 'reduce' of 'numpy.ufunc' objects>)
24			0.1184		0.004933		0.1184		0.004933		~0(<built-in method scipy.sparse._sparsetools.get_csr_submatrix>)
30			0.1079		0.003596		0.1085		0.003616		_utils.py:47(sparse_mean_variance_axis)
75			0.07802		0.00104		0.07802		0.00104		~0(<built-in method numpy.zeros>)
366			0.07657		0.0002092		0.07708		0.0002106		missing.py:261(_isna_array)
67			0.05807		0.0008668		0.05807		0.0008668		~0(<method 'argsort' of 'numpy.ndarray' objects>)
1			0.05747		0.05747		2.465		2.465		_rank_genes_groups.py:174(_basic_stats)
16			0.05085		0.003178		11.66		0.7287		_rank_genes_groups.py:277(wilcoxon)
24			0.03967		0.001653		0.03967		0.001653		~0(<built-in method scipy.sparse._sparsetools.csr_todense>)
1			0.03775		0.03775		0.03775		0.03775		~0(<built-in method scipy.sparse._sparsetools.csr_eliminate_zeros>)
1			0.03247		0.03247		19.28		19.28		_rank_genes_groups.py:398(compute_statistics)
360			0.02029		5.636e-05		0.3671		0.00102		frame.py:11462(_reduce)
2440			0.01726		7.076e-06		0.01728		7.083e-06		~0(<built-in method __new__ of type object at 0x10522b330>)
53290/53252			0.0115		2.159e-07		0.02308		4.335e-07		~0(<built-in method builtins.isinstance>)
7			0.01055		0.001507		0.01057		0.00151		~0(<method 'acquire' of '_thread.lock' objects>)
15			0.007104		0.0004736		0.007104		0.0004736		_continuous_distats.py:370(_norm_cdf)
24			0.005682		0.0002367		0.2439		0.01016		_rank_genes_groups.py:70(_clambda>)
488			0.005585		1.144e-05		0.005585		1.144e-05		~0(<method 'nonzero' of 'numpy.ndarray' objects>)
360			0.005211		1.447e-05		0.0912		0.0002533		nanops.py:1499(_maybe_null_out)
382			0.005043		1.32e-05		0.02591		6.782e-05		base.py:475(__new__)
23			0.004954		0.0002154		0.004954		0.0002154		~0(<method 'read' of '_io.BufferedReader' objects>)

Runtime vs Dataset size analysis -

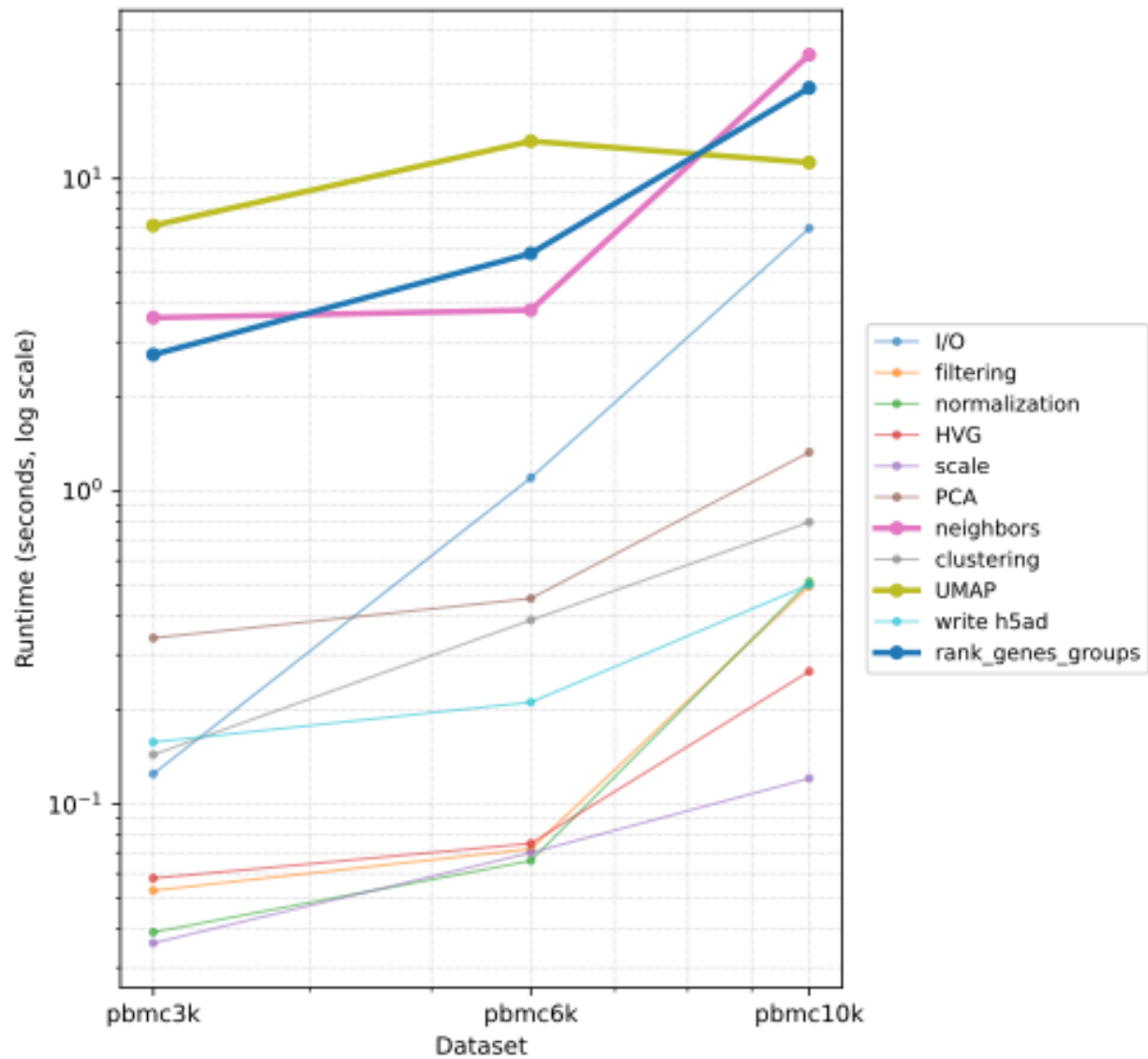


Figure 1 - Instrumented runtime by section vs Dataset size

Neighbours, UMAP, and rank_genes_groups are the bottlenecks - 93% to 97% of runtime at all three sizes. They scale badly as compared to even linear with cell count so at 20k cells they will take approx 70-120 seconds.

Slurm Batch Scripts -

```
$ run_pbmc3k.sh
1  #!/bin/bash -l
2  #SBATCH -J scanpy_pbmc3k
3  #SBATCH -p overflow
4  #SBATCH --mem=2G
5  #SBATCH --cpus-per-task=2
6  #SBATCH --time=00:10:00
7  #SBATCH --output=./slurm_outputs/%j.out
8
9  module load bmi/python-3.13.5
10 conda activate bmi500
11
12 cd $SLURM_SUBMIT_DIR
13
14 /usr/bin/time -v python scanpy_pbmc.py --data-dir data --data-set pbmc3k --out-dir data
```

```
$ run_pbmc6k.sh
1  #!/bin/bash -l
2  #SBATCH -J scanpy_pbmc6k
3  #SBATCH -p overflow
4  #SBATCH --mem=2G
5  #SBATCH --cpus-per-task=2
6  #SBATCH --time=00:10:00
7  #SBATCH --output=./slurm_outputs/%j.out
8
9  module load bmi/python-3.13.5
10 conda activate bmi500
11
12 cd $SLURM_SUBMIT_DIR
13
14 /usr/bin/time -v python scanpy_pbmc.py --data-dir data --data-set pbmc6k --out-dir data
```

```
$ run_pbmc10k.sh
1  #!/bin/bash -l
2  #SBATCH -J scanpy_pbmc10k
3  #SBATCH -p overflow
4  #SBATCH --mem=4G
5  #SBATCH --cpus-per-task=2
6  #SBATCH --time=00:15:00
7  #SBATCH --output=./slurm_outputs/%j.out
8
9  module load bmi/python-3.13.5
10 conda activate bmi500
11 |
12 cd $SLURM_SUBMIT_DIR
13
14 /usr/bin/time -v python scanpy_pbmc.py --data-dir data --data-set pbmc10k --out-dir data
```

Repo - <https://github.com/ShashankChauhan01/bmi500>